Assignment Report

Student: Denisenko Sofiia 狄苏菲
Student id: 1820249051
Course: Computer Organization and Architecture

Assignment №5

1.

The frequency of a computer's CPU is 8MHz, if each machine cycle contains 4 clock cycles, the average instruction execution speed is 0.8MIPS (million instructions per second).
(1) What is the average instruction cycle?
(2) How many machine cycles does each instruction cycle contains on average?
(3) If you want to get an instruction execution speed as 400,000 times/second, what is the CPU frequency should be used?

Comp. Org. & Arch.

Assignment 5.

1. CPU freq: 8 MHz
Machine cycle contains 4 clock cycles
Avg. instr. execution speed: 0,8 MIPS

(1) $\text{Avg. instr. cycle} = \frac{\text{Time / instr.}}{\text{Time / clock cycle}}$

$$\left.\begin{aligned}\text{Time / Instr.} &= \frac{1}{\text{instr. exec. speed}} \\ \text{Time / clock cycle} &= \frac{1}{\text{CPU freq}}\end{aligned}\right\} \Rightarrow$$

$$\text{Avg. instr. cycle} = \frac{\text{CPU freq}}{\text{instr. exec. speed}} = \frac{8}{0{,}8} = $$
$= 10$ clock cycles

(2) $\text{Machine cycles / instr} = \frac{\text{Avg. instr cycle}}{\text{Clock cycles / machine cycle}} = \frac{10}{4} = 2{,}5$

(3) New avg. instr. execution speed: 0,4 MIPS
CPU freq - ?

$$\text{New time / instr} = \frac{1}{\text{New exec. speed}} = \frac{1}{0{,}4\cdot 10^{6}} =$$
$= 2{,}5 \cdot 10^{-6}$ sec/instr.

$$\text{New time / clock cycle} = \frac{\text{New time / instr}}{\text{Avg. instr cycle}} = \frac{2{,}5\cdot 10^{-6}}{10} = 0{,}25\cdot 10^{-6} \text{ sec / clock cycle}$$

$$\text{New CPU freq.} = \frac{1}{\text{New time / clock cycle}} = \frac{1}{0{,}25\cdot 10^{-6}} =$$
$= 4$ MHz

Answer: (1) 10 clock cycles
(2) 2,5 machine cycles
(3) 4 MHz

2.

Instructions and data are both stored in the main memory, how to identify the information fetched from the main memory is the instruction or data?

There are multiple ways:

1. By using **Program Counter (PC)** and **Instruction Fetch**:

The Program Counter (PC): This is a special register within the CPU that holds the memory address of the next instruction to be executed.
Sequential Execution: The CPU fetches instructions from memory sequentially based on the value of the PC.
Instruction Fetch Cycle: This dedicated phase of the CPU's execution cycle is solely responsible for obtaining instructions from memory. It does this using the PC.

2. By **Instruction Decode** and **Execution**:

Instruction Register (IR): After an instruction is fetched from memory, it is loaded into the Instruction Register (IR) within the CPU.
Instruction Decode: The CPU's control unit analyzes the contents of the IR. It breaks down the instruction's binary code into opcode, operand(s), and other control signals.
Operand Fetch: If the instruction requires data (operands), the CPU will use the information from the decoded instruction to calculate the memory address of the data and fetch it.
Execution: Once the operands are available, the CPU executes the instruction.

3. By **Context** and **Timing**:

Instruction Fetch Timing: Instructions are always fetched during the "fetch" phase of the CPU cycle. The CPU knows from the state whether it is fetching an instruction or fetching data.
Data Fetch Timing: Data is fetched only when the decoded instruction dictates, as determined in the decode and execution phases.

4. By **Addressing Modes**:

Addressing Modes provide various ways for the CPU to calculate the memory addresses of data required by an instruction. These modes are built into the instruction set and dictate if the operand is a memory address, an immediate value, or a register, and if it refers to data or an address.

3.

Write the execution process of the instruction **ADD (R1), (R2)+**. This instruction is to perform addition. The two operands' addresses are in the register R1 and R2 respectively. After finishing the addition, the content in R2 will be increased by 1, as the address of the operation result.

①The process should include the instruction fetching process and the determining subsequent instruction address process;

② You can use the register C and D to store the operands for ALU.

Execution Process:

Phase 1: Instruction Fetch

1) PC (Program Counter) -> MAR (Memory Address Register)
2) Memory Read (Memory[MAR] -> MDR (Memory Data Register))
3) MDR -> IR (Instruction Register)
4) PC Increment. The PC is incremented by the size of the current instruction (e.g., by 2 or 4 depending on the instruction size). This makes the PC point to the next instruction in the program, preparing for the next fetch cycle.

Phase 2: Instruction Decode

1) IR Decoded.

The control unit decodes the instruction in the IR: ADD (R1), (R2)+. It understands that this is an additional instruction with two memory operands and that the register R2 needs to be incremented after the addition is done. The control unit sets the appropriate control signals for data retrieval and ALU operations.

Phase 3: Operand Fetch (First Operand)

1) R1 -> MAR
2) Memory Read (Memory[MAR] -> MDR)
3) MDR -> C

Phase 4: Operand Fetch (Second Operand)

1) R2 -> MAR
2) Memory Read (Memory[MAR] -> MDR)
3) MDR -> D

Phase 5: Execution (ALU Operation)

1) C + D -> ACC. The values in registers C and D are fed as input to the Arithmetic Logic Unit (ALU). The ALU performs the addition operation and stores the sum into the Accumulator register ACC.

Phase 6: Operand Increment

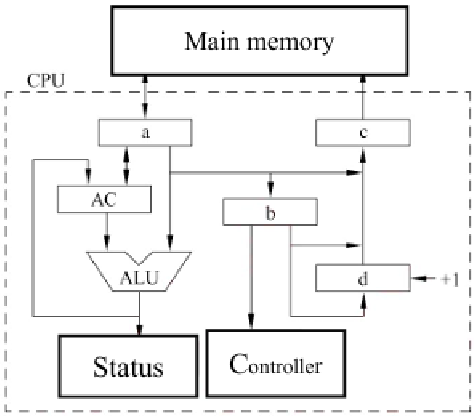
1) R2 + 1 -> R2

Phase 7: Determining the Address of the Next Instruction

1) PC -> MAR
2) Memory Read (Memory[MAR] -> MDR
3) MDR -> IR. The instruction in MDR is copied to the Instruction register IR. This sets the stage for the next instruction cycle.

4.

The structure of CPU is as shown figure. There is an accumulating register AC, a status register and 4 other registers. The connection between each component represents the data path, and the arrow represents the direction of information transmission.



(1) What are the names of these 4 registers?

a: TheMemory Data Register (MDR).

b: The Program Counter (PC).

c: The Memory Address Register (MAR).

d: TheInstruction Register (IR).

(2) Briefly describe the data path of the instruction from the main memory to the controller.

1. Address Retrieval: The Program Counter (PC), register ‘b’, holds the address of the next instruction in memory.
2. MAR Load: The address in the PC is loaded into the Memory Address Register (MAR), which is register ‘c’.

3. Memory Fetch: The main memory is accessed at the address held in the MAR.
4. MDR Load: The instruction fetched from memory is placed in the Memory Data Register (MDR), which is register ‘a’.
5. Instruction Transfer: The instruction from MDR is transferred to the Instruction Register (IR), which is register ‘d’.
6. Control Unit Action: The control unit reads the instruction from the IR to understand the opcode and other instruction details, determining which operations to perform next.

(3) Briefly describe the data path of the data between the main memory and ALU, including reading and writing.

Data Read from Main Memory to ALU:

1) Address Calculation: The CPU calculates the memory address where the data to be read is stored.
2) MAR Load: The calculated address is loaded into the MAR (register ‘c’).
3) Memory Fetch: The main memory is accessed at the address stored in the MAR.
4) MDR Load: The data from the accessed memory location is loaded into the MDR (register ‘a’).
5) MDR to AC: The data in the MDR is copied to the Accumulator register AC.
6) AC to ALU: The data in the AC register is passed to the Arithmetic Logic Unit (ALU).

Data Write from ALU to Main Memory:

1) ALU Operation: The ALU performs its operation, and the result is stored in the AC register.
2) MDR Load The result in the AC is copied to MDR (register ‘a’).
3) Address Load: The CPU determines the memory address where the result should be stored.
4) MAR Load: The address of where the result is to be stored is copied to MAR (register ‘c’).
5) Memory Write: The content of the MDR (result of the operation) is written to the main memory at the address given by the MAR.

5.

What is the micro-command and micro-operation? What is the relation between microprogram and machine instruction? What is the relation between microprogram and program?

1) **Micro-operation**. A micro-operation is the most basic and elementary operation that can be performed by a CPU's hardware. A machine instruction can be decomposed into a sequence of micro-operations. These microoperations are the most basic and nondecomposable operations in a computer.

Micro-commands are the smallest unit of a control sequence, such as open or close the potential signal of a certain control door. Therefore, the micro-commands control the computer to complete basic micro-operations.

2) Micro-command is the control signal of microoperation, and micro-operation is the operation process of micro-command.
3) Each machine instruction corresponds to a section of "program". We call these "programs" as microprograms of instructions. Store these microprograms into a read-only memory (control memory).

6.

The machine has 8 micro-instructions as I1~I8, and the micro-commands for each micro-instruction is as the following table, where a~j respectively represent 10 different types of micro-command signals. Assuming that the operation control field of a micro-instruction is 8 bits, please design a format of the operation control fields for all micro-instructions (note that some micro-commands are compatible), and give the coding of the control field for all the micro-instructions.

Micro-instructions	Micro-commands									
	a	b	c	d	e	f	g	h	i	j
I_1	√	√	√	√	√					
I_2	√			√		√	√			
I_3		√						√		
I_4			√							
I_5			√		√		√		√	
I_6	√							√		√
I_7			√	√				√		
I_8	√	√						√		

Analysis of Compatibility

- 'a' and 'f' are never used together.
- 'b', 'i', and 'j' are never used together.
- 'c' and 'g' are never used together.
- 'd' is not used with 'h'.

Designing the 8-bit Control Field Format

Based on the compatibility, we can come up with the following format:

- Bit 0: a/f (0 = a, 1 = f)
- Bit 1: b/i/j (00 = b, 01 = i, 10 = j)
- Bit 2: c/g (0 = c, 1 = g)
- Bit 3: d/h (0 = d, 1 = h)
- Bit 4: e
- Bits 5-7: not used for micro-command, set them to '0'

Encoding Micro-Instructions

I1: a, b, c, d, e = 00001101 = 0x0D

I2: a, d, f, g = 11100001 = 0xE1

I3: b, h = 01010000 = 0x50

I4: c, e = 00001100 = 0x0C

I5: c, g, i = 01000110 = 0x46

I6: a, h, j = 10110001 = 0xB1

I7: c, d, h = 01010100 = 0x54

I8: a, b, h = 01010001 = 0x51

7.

Assuming that a bus transmits a 4-byte data in one clock cycle, the clock frequency of the bus is 33MHz, what is the bus bandwidth?
If the data bus width is changed to 64 bits, one clock cycle can transmit data twice, the bus clock frequency is 66MHz, what is the bus bandwidth?

7. 4 bytes/clock cycle (width)
Clock freq: 33MHz

(1) Bandwidth = $\frac{\text{width} \times \text{freq}}{N \text{ of cycles}}$

$= \frac{4 \times 33 \times 10^6}{1} = \underline{132 \text{ MB/s}}$

(2) New width → 64 bits = 8 bytes
One clock cycle = twice data
Clock freq = 66 MHz

1 clock cycle = 2 data → data transfer = $\frac{1}{2}$ clock cycle

Bandwidth = $\frac{\text{Width} \times \text{freq}}{N \text{ of cycles}} = \frac{8 \times 66 \times 10^6}{\frac{1}{2}} =$

$= 8 \times 66 \times 10^6 \times 2 = \underline{1056 \text{ MB/S}}$

Answer: (1) 132 MB/S
(2) 1056 MB/S

8.

The clock frequency of a certain bus is 66MHz, and the bus width is 64 bits. In 7 clock cycles, it can transmit a data block with 6 words.

(1) What is the data transfer rate of the bus?

(2) If the size of the data block is not changed, but the clock frequency is halved, what is the data transfer rate of the bus?

8 Clock freq : 66MHz

Bus width : 64 bits = 8 bytes

In 7 cycles → block with 6 words

(1) Data transfer rate =

$$= \frac{\text{Width} \cdot \text{freq} \cdot \text{N of words/block}}{\text{clock cycles/block}} =$$

$$= \frac{8 \cdot 66 \cdot 10^6 \cdot 6}{7} \approx 452{,}57 \text{ MB/S}$$

(2) New clock freq : 33 MHz

= old ÷ 2

$$\text{New data transfer rate} = \frac{\text{Width} \cdot \frac{\text{Old Freq}}{2} \cdot \text{N of words/block}}{\text{Clock cycles/block}} =$$

$$= \frac{\text{Old Data transfer rate}}{2} = \frac{452{,}57}{2} =$$

$\approx 226{,}29$ MB/S

Answer: (1) 452,57 MB/S

(2) 226,29 MB/S

9.

A certain disk group has 6 disks, each disk has two recording surfaces (platters). The storage area has inner diameter as 22cm, outer diameter as 33cm, track density as 40 tracks/cm, bit density as 400b/cm, and rotation speed as 2400r/min.

(1) How many platters are available?
(2) How many cylinders are there in total?
(3) What is the total storage capacity of the entire disk group? (You can just consider the length of innermost cycle as the length of all track)
(4) What is the data transfer rate (which is the (capacity of one track)÷(rotation time for one round))?
(5) If the length of a file exceeds the capacity of one track, should it be recorded on the same platter or on the same cylinder?
(6) How to indicate the disk address?

9. N of disks: 6
each has 2 platters
Inner diameter: 22 cm ; Outer: 33 cm
track density: 40 tracks/cm
bit density: 400 bits/cm
rotation speed: 2400 rotation/minute

(1) N of platters = N of disks × 2 = 6×2 =
= 12 platters

(2) Width = Outer radius - Inner radius =
= (Outer diameter - Inner diameter) /2 =
= (33-22)/2 = 11 : 2 = 5,5 cm
N of cylinders = width × track density =
= 5,5 × 400 = 220 cylinders

(3) Total storage capacity - ?
L of track = L of innermost circle = 2π·Inner Radius = π· Inner Diameter = π× 22 ≈
≈ 69,1 cm
Capacity / track = Track Length × Bit density =
= 69,1 · 400 ≈ 27640 bits = 3455 bytes
Capacity/ platter = Capacity/ track × N of cylinders = 3455 × 220 = 760100 bytes ≈ 742,285 KB
Total capacity = Capacity/platter × N of platters =
= 742,285 × 12 ≈ 8907,42 KB = 8,7 MB

(4) Data transfer rate -?
Capacity / track = 3455 bytes
Time / round = $\frac{1 \text{ minute}}{2400 \text{ rotations}} = \frac{60}{2400}$ =
= 0,025 sec / rotation
Data transfer rate = $\frac{\text{Capacity/track}}{\text{Time / round}}$ =
= $\frac{3455}{0,025}$ = 138200 bytes/sec ≈ 135 KB/s

(5) On the same cylinder
Each platter has one magnetic head, so reading /writing information on different platter (same cylinder) can be fast

(6) Disc address = [Drive Number, Cylinder (track) number, platter (head) number, sector number]